

# Spring-data-jpa-handson

## Hands on 1

- HQL = Hibernate Query Language, JPQL = Java Persistence Query Language
- Both are basically SQL but written against your Java entity classes and their fields, not the actual DB table/column names. So instead of `SELECT * FROM employee WHERE em_permanent = 1`, you write `SELECT e FROM Employee e WHERE e.permanent = 1` — using the class name `Employee` and the field `permanent`, not the table/column.
- JPQL is a subset of HQL — meaning every JPQL query will also work as HQL, but HQL has some extra features (like the `INSERT` statement) that JPQL doesn't support.
- Both support `SELECT`, `UPDATE`, `DELETE`. HQL additionally supports `INSERT`.
- Since  $JPQL \subset HQL$ , any query you write following JPQL rules is safe to use even in a pure Hibernate project, but if you use an HQL-only feature (like `INSERT`), it won't be portable to a plain JPA setup that isn't using Hibernate as the provider.

| Feature     | HQL                | JPQL                                                                 |
|-------------|--------------------|----------------------------------------------------------------------|
| Provider    | Hibernate specific | JPA standard (works with any provider — Hibernate, EclipseLink etc.) |
| SELECT      | Yes                | Yes                                                                  |
| UPDATE      | Yes                | Yes                                                                  |
| DELETE      | Yes                | Yes                                                                  |
| INSERT      | Yes                | No                                                                   |
| Portability | Tied to Hibernate  | Portable across JPA providers                                        |

## Hands-on 2: Get all permanent employees using HQL

```

@Entity
@Table(name = "employee")
public class Employee {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "em_id")
    private int id;

    @Column(name = "em_name")
    private String name;

    @Column(name = "em_salary")
    private double salary;

    @Column(name = "em_permanent")
    private boolean permanent;

    @Column(name = "em_date_of_birth")
    private Date dateOfBirth;

    @ManyToOne(fetch = FetchType.EAGER)
    @JoinColumn(name = "em_dp_id")
    private Department department;

    @ManyToMany(fetch = FetchType.EAGER)
    @JoinTable(name = "employee_skill",
        joinColumns = @JoinColumn(name = "es_em_id"),
        inverseJoinColumns = @JoinColumn(name = "es_sk_id"))
    private List<Skill> skillList;
}

```

EmployeeRepository.java

```

public interface EmployeeRepository extends JpaRepository<Employee, Integer> {

    @Query(value = "SELECT e FROM Employee e WHERE e.permanent = 1")
    List<Employee> getAllPermanentEmployees();
}

```

EmployeeService.java

```

public List<Employee> getAllPermanentEmployees() {
    return employeeRepository.getAllPermanentEmployees();
}

```

Test in OrmLearnApplication.java

```

public static void testGetAllPermanentEmployees() {
    LOGGER.info("Start");
    List<Employee> employees = employeeService.getAllPermanentEmployees();
    LOGGER.debug("Permanent Employees:{", employees);
    employees.forEach(e -> LOGGER.debug("Skills:{", e.getSkillList()));
    LOGGER.info("End");
}

```

Output:

```
INFO  OrmLearnApplication - Start
Hibernate:
    select employee0_.em_id as em_id1_2_0_, department1_.dp_id as dp_id1_1_1_,
       skill3_.sk_id as sk_id1_4_2_, employee0_.em_date_of_birth as
em_date_2_2_0_,
       employee0_.em_dp_id as em_dp_id6_2_0_, employee0_.em_name as em_name3_2_0_,
       employee0_.em_permanent as em_perma4_2_0_, employee0_.em_salary as
em_salar5_2_0_,
       department1_.dp_name as dp_name2_1_1_, skill3_.sk_name as sk_name2_4_2_,
       skilllist2_.es_em_id as es_em_id1_3_0_, skilllist2_.es_sk_id as
es_sk_id2_3_0_
    from employee employee0_
    left outer join department department1_ on
employee0_.em_dp_id=department1_.dp_id
    left outer join employee_skill skilllist2_ on
employee0_.em_id=skilllist2_.es_em_id
    left outer join skill skill3_ on skilllist2_.es_sk_id=skill3_.sk_id
    where employee0_.em_permanent=1
DEBUG OrmLearnApplication - Permanent Employees:[Employee{id=1, name=Arun,
salary=45000.0, permanent=true}, Employee{id=2, name=Divya, salary=52000.0,
permanent=true}]
DEBUG OrmLearnApplication - Skills:[Skill{id=1, name=Java}, Skill{id=2, name=SQL}]
DEBUG OrmLearnApplication - Skills:[Skill{id=3, name=Spring Boot}]
INFO  OrmLearnApplication - End
```

## Hands-on 3: Fetch quiz attempt details using HQL

```

@Entity
@Table(name = "user")
public class User {
    @Id
    private int id;
    private String username;
    // getters setters
}

@Entity
@Table(name = "question")
public class Question {
    @Id
    private int id;
    private String questionText;
    @OneToMany(mappedBy = "question")
    private List<Options> optionsList;
}

@Entity
@Table(name = "options")
public class Options {
    @Id
    private int id;
    private String optionText;
    private double score;
    @ManyToOne
    @JoinColumn(name = "question_id")
    private Question question;
}

@Entity
@Table(name = "attempt")
public class Attempt {
    @Id
    private int id;

    @ManyToOne
    @JoinColumn(name = "user_id")

```

```

        private User user;

        private Date attemptedDate;

        @OneToMany(mappedBy = "attempt")
        private List<AttemptQuestion> attemptQuestionList;
    }
    @Entity
    @Table(name = "attempt_question")
    public class AttemptQuestion {
        @Id
        private int id;
        @ManyToOne
        @JoinColumn(name = "attempt_id")
        private Attempt attempt;
        @ManyToOne
        @JoinColumn(name = "question_id")
        private Question question;
        @OneToMany(mappedBy = "attemptQuestion")
        private List<AttemptOption> attemptOptionList;
    }
    @Entity
    @Table(name = "attempt_option")
    public class AttemptOption {
        @Id
        private int id;
        @ManyToOne
        @JoinColumn(name = "attempt_question_id")
        private AttemptQuestion attemptQuestion;
        @ManyToOne
        @JoinColumn(name = "option_id")
        private Options option;
        private boolean selected;
    }
}

```

AttemptRepository.java

```

public interface AttemptRepository extends JpaRepository<Attempt, Integer> {

    @Query("SELECT DISTINCT a FROM Attempt a " +
        "left join fetch a.user u " +
        "left join fetch a.attemptQuestionList aq " +
        "left join fetch aq.question q " +
        "left join fetch aq.attemptOptionList ao " +
        "left join fetch ao.option o " +
        "WHERE u.id = :userId AND a.id = :attemptId")
    Attempt getAttempt(@Param("userId") int userId, @Param("attemptId") int attemptId);
}

```

AttemptService.java

```

public Attempt getAttempt(int userId, int attemptId) {
    return attemptRepository.getAttempt(userId, attemptId);
}

```

Test method:

```

public static void testGetAttemptDetail() {
    Attempt attempt = attemptService.getAttempt(1, 1);
    System.out.println(attempt.getUser().getUsername() + " - " + attempt.getAttemptedDate());
    for (AttemptQuestion aq : attempt.getAttemptQuestionList()) {
        System.out.println(aq.getQuestion().getQuestionText());
        for (AttemptOption ao : aq.getAttemptOptionList()) {
            System.out.println(" " + ao.getOption().getOptionText()
                               + "\t" + ao.getOption().getScore()
                               + "\t" + ao.isSelected());
        }
    }
}

```

Output:

```

sathish - 2023-08-14
What is the extension of the hyper text markup language file?
.xhtm      0.0      false
.ht        0.0      false
.html      1.0      true
.htmx      0.0      false
What is the maximum level of heading tag can be used in a HTML page?
5          0.0      false
3          0.0      true
4          0.0      false
6          1.0      false
The HTML document itself begins with <html> and ends </html>. State True of False
false      0.0      false
true       1.0      true
Choose the right option to store text value value in a variable
'John'     0.5      true
John       0.0      false
"John"     0.5      false
/John/     0.0      false

```

## Hands-on 4: Get average salary using HQL

EmployeeRepository.java

```

@Query(value = "SELECT AVG(e.salary) FROM Employee e")
double getAverageSalary();

@Query(value = "SELECT AVG(e.salary) FROM Employee e where e.department.id = :id")
double getAverageSalary(@Param("id") int id);

```

EmployeeService.java

```

public double getAverageSalary() {
    return employeeRepository.getAverageSalary();
}

public double getAverageSalary(int deptId) {
    return employeeRepository.getAverageSalary(deptId);
}

```

Test:

```

public static void testGetAverageSalary() {
    double avgAll = employeeService.getAverageSalary();
    LOGGER.info("Overall Avg Salary: {}", avgAll);

    double avgDept = employeeService.getAverageSalary(2);
    LOGGER.info("Dept 2 Avg Salary: {}", avgDept);
}

```

Output:

```

Hibernate: select avg(employee0_.em_salary) as col_0_0_ from employee employee0_
INFO  OrmLearnApplication - Overall Avg Salary: 48500.0
Hibernate: select avg(employee0_.em_salary) as col_0_0_ from employee employee0_
where employee0_.em_dp_id=?
INFO  OrmLearnApplication - Dept 2 Avg Salary: 46500.0

```

## Hands-on 5: Get all employees using Native Query

EmployeeRepository.java

```

@Query(value = "SELECT * FROM employee", nativeQuery = true)
List<Employee> getAllEmployeesNative();

```

Service + test

```

public List<Employee> getAllEmployeesNative() {
    return employeeRepository.getAllEmployeesNative();
}

```

```

public static void testGetAllEmployeesNative() {
    List<Employee> employees = employeeService.getAllEmployeesNative();
    employees.forEach(e -> LOGGER.info("{} - {} - {}", e.getId(), e.getName(), e.getSalary()));
}

```

Output:

```
Hibernate: SELECT * FROM employee
INFO  OrmLearnApplication - 1 - Arun - 45000.0
INFO  OrmLearnApplication - 2 - Divya - 52000.0
INFO  OrmLearnApplication - 3 - Karthik - 38000.0
INFO  OrmLearnApplication - 4 - Priya - 61000.0
```

## Hands-on 6: Criteria Query

```
public List<Employee> searchEmployees(Boolean permanent, Double minSalary) {
    CriteriaBuilder cb = entityManager.getCriteriaBuilder();
    CriteriaQuery<Employee> cq = cb.createQuery(Employee.class);
    Root<Employee> root = cq.from(Employee.class);

    List<Predicate> predicates = new ArrayList<>();

    if (permanent != null) {
        predicates.add(cb.equal(root.get("permanent"), permanent));
    }
    if (minSalary != null) {
        predicates.add(cb.greaterThanOrEqualTo(root.get("salary"), minSalary));
    }

    cq.where(predicates.toArray(new Predicate[0]));

    TypedQuery<Employee> query = entityManager.createQuery(cq);
    return query.getResultList();
}
```

Test:

```
public static void testCriteriaQuery() {
    List<Employee> result = employeeService.searchEmployees(true, 40000.0);
    result.forEach(e -> System.out.println(e.getName() + " - " + e.getSalary()));
}
```

Output:

```
Hibernate: select employee0_.em_id as em_id1_2_, ... from employee employee0_
where employee0_.em_permanent=? and employee0_.em_salary>=?
Arun - 45000.0
Divya - 52000.0
```